



SCARLET

System Container Application Release, Lifecycle & Environment Tool

DOCUMENTO DI ARCHITETTURA

Architettura e funzionamento

Come è fatto SCARLET e perché funziona così

SCARLET non è un esecutore di script di deploy: è un control plane che tiene, per ogni applicazione e per ogni server, lo stato desiderato e quello osservato, e riduce la distanza fra i due. Questo documento descrive i livelli del software, gli oggetti di dominio, l'anatomia completa di un rilascio, l'attivazione atomica con rollback, gli adattatori di runtime, il contratto del pacchetto e i controlli di sicurezza, con i diagrammi che li mettono in relazione.

IN QUESTO DOCUMENTO

- Il modello a stato desiderato e stato effettivo, e come nasce la divergenza
- I sei livelli del software e la sola direzione di dipendenza ammessa
- I tredici passi di un rilascio, il punto di non ritorno e il rollback automatico
- L'attivazione atomica della release e perché il ritorno indietro è immediato
- I controlli di sicurezza: comandi in allow-list, chiavi host, permessi di produzione
- Sette diagrammi che mettono in relazione componenti, passi e stati

DESTINATARI	Architetti, sviluppatori, responsabili tecnici, sicurezza
VERSIONE	SCARLET 1.0.0
DATA	14/09/2026

Indice

1. SCARLET in una pagina

2. Il modello: un control plane

Stato desiderato, stato effettivo, divergenza

3. Componenti e topologia

4. I livelli del software

5. Gli oggetti di dominio

6. Anatomia di un rilascio

Dal wizard alla coda

I tredici passi del piano

La macchina a stati

7. Attivazione atomica e rollback

8. Adattatori di runtime

9. Il contratto del pacchetto

10. Sicurezza per progetto

Come nasce un comando remoto

Controlli sulla produzione

11. Riconciliazione e divergenza

12. Job asincroni, code e lock

13. Modello dei dati

14. Osservabilità e tracciabilità

15. Punti di estensione

1. SCARLET in una pagina

SCARLET gestisce il ciclo di vita delle applicazioni su server Oracle Linux remoti: le installa, le avvia, le ferma, le aggiorna, le riporta alla versione precedente e ne sorveglia lo stato. Lo fa via SSH, senza installare nulla sui server gestiti.

La differenza rispetto a uno script di deploy sta in una scelta di fondo: SCARLET non è un esecutore di comandi, è un **control plane**. Per ogni coppia applicazione-server conserva due fotografie, quella che gli operatori hanno chiesto e quella osservata sul campo, e il suo compito è ridurre la distanza fra le due. Tutto il resto, dalla pianificazione dei passi alla gestione degli errori, discende da qui.

Principio	Come si traduce nel codice
Stato dichiarato, non comandi	ogni istanza applicativa ha uno stato desiderato e uno effettivo; il motore calcola i passi necessari per allinearli
Un solo modo di raggiungere l'host	tutto passa dal livello SSH, che accetta solo liste di argomenti con eseguibile in allow-list; non esiste una funzione per eseguire testo libero
Piano prima dell'esecuzione	il pianificatore produce un DeploymentPlan salvato nel database; l'esecuzione registra ogni passo, quindi è ricostruibile a posteriori
Il runtime è un dettaglio	Docker, Podman e Kubernetes stanno dietro la stessa interfaccia; aggiungerne uno significa scrivere una classe, non toccare il motore
La produzione si difende da sola	permessi dedicati, frase da digitare, motivazione obbligatoria, approvazione di una seconda persona, lock per applicazione e host
Tutto lascia traccia	audit in sola aggiunta, eventi di sicurezza, identificativo di richiesta propagato dal browser fino al comando remoto

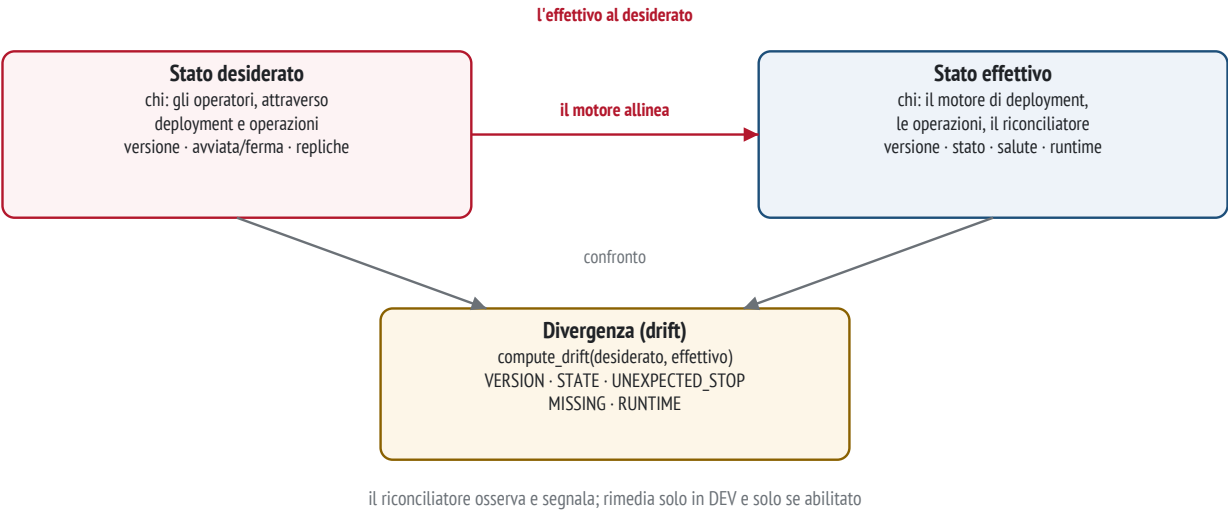


Figura 1 – I due stati e la divergenza fra loro: è il cuore del modello.

2. Il modello: un control plane

I server remoti sono il piano di esecuzione; SCARLET è il piano di controllo. Per ogni combinazione di applicazione e server esiste una riga, l'istanza applicativa, che tiene insieme le tre informazioni che contano.

	Campi	Chi li scrive
Stato desiderato	<code>desired_version</code> , <code>desired_state</code> (RUNNING, STOPPED, ABSENT), <code>desired_replicas</code>	gli operatori, attraverso i deployment e le operazioni di ciclo di vita
Stato effettivo	<code>actual_version</code> , <code>actual_state</code> , <code>actual_replicas</code> , <code>actual_runtime</code> , <code>health_status</code> , <code>actual_observed_at</code>	il motore di deployment, le operazioni di ciclo di vita, il riconciliatore
Divergenza	<code>drift_detected</code> , <code>drift_type</code> , <code>drift_details</code>	la funzione <code>compute_drift(desiderato, effettivo)</code>

Stato desiderato, stato effettivo, divergenza

La divergenza non è un errore: è un'informazione. Dice che il mondo reale si è allontanato da ciò che era stato deciso, e il motivo può essere legittimo quanto un riavvio del server o grave quanto un container fermato a mano in produzione. SCARLET la classifica in cinque tipi, così che l'operatore sappia subito che cosa sta guardando.

Tipo	Che cosa è successo	Reazione tipica
VERSION	sull'host gira una versione diversa da quella desiderata	ridistribuire la versione attesa oppure accettare quella presente
STATE	l'applicazione dovrebbe essere avviata ed è ferma, o viceversa	avviare o fermare dalla pagina dell'applicazione
UNEXPECTED_STOP	il container è uscito senza che nessuno lo abbia chiesto	leggere i log: quasi sempre è l'applicazione a essere terminata
MISSING	il container non esiste più sull'host	riavviare il rilascio; verificare linger e politiche di riavvio
RUNTIME	l'applicazione gira con un runtime diverso da quello registrato	allineare la scheda dell'host o l'installazione

Osservare non significa correggere

Il riconciliatore **osserva sempre** ma **interviene solo se glielo si chiede**, e solo negli ambienti non di produzione. È una scelta deliberata: un sistema che corregge da solo la produzione nasconde gli incidenti invece di renderli visibili.

3. Componenti e topologia

Il control plane è un insieme di processi che girano come container su una sola macchina. Gli unici due componenti che parlano con il mondo esterno sono nginx, verso gli operatori, e il worker, verso i server gestiti.

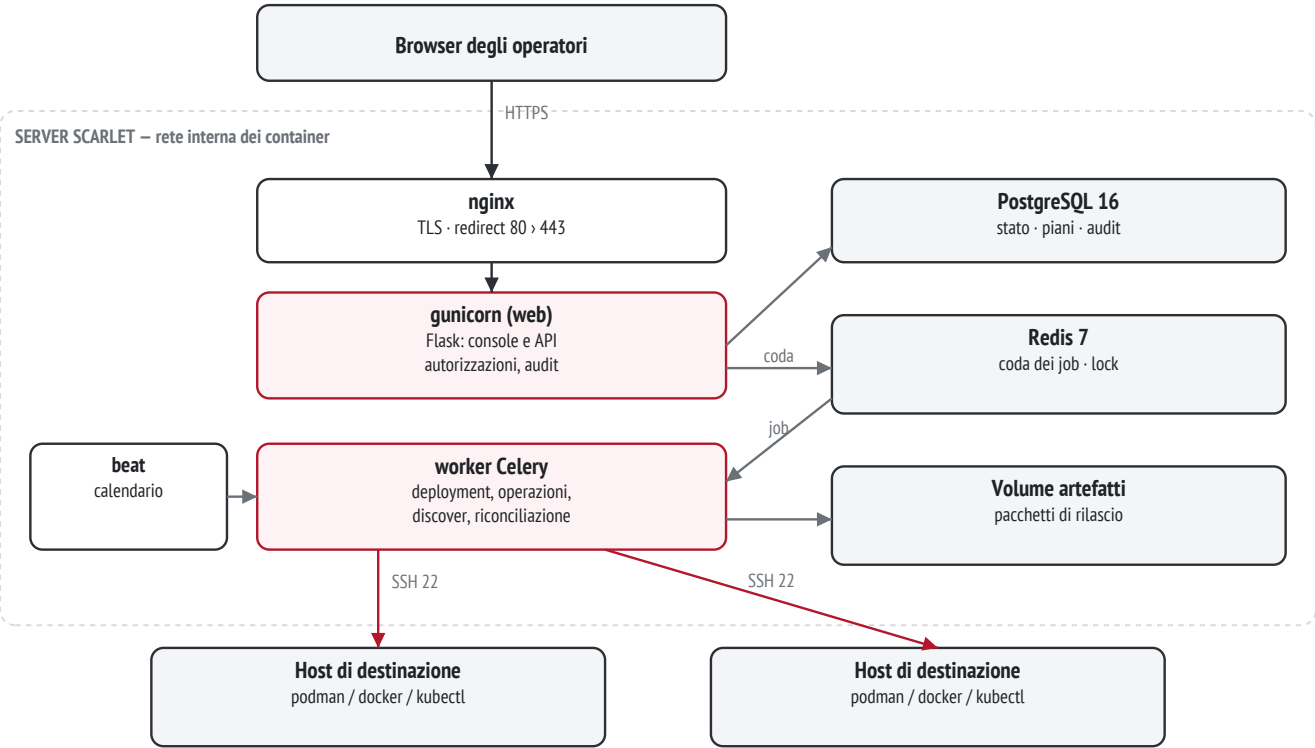
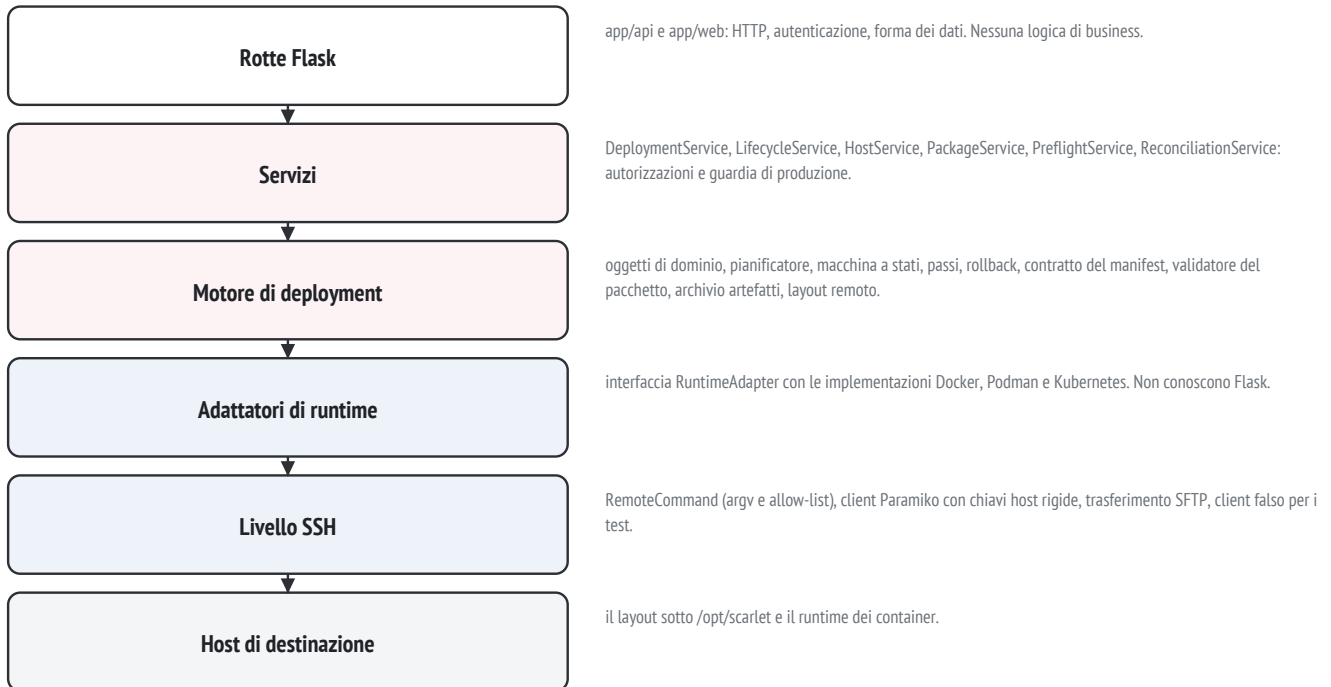


Figura 2 – I processi del control plane e i due soli flussi verso l'esterno.

Componente	Ruolo	Perché è separato
nginx	termina il TLS, reindirizza la 80 sulla 443, applica il limite di dimensione degli upload	l'applicazione non gestisce mai TLS né file di grandi dimensioni in ingresso
gunicorn (web)	serve la console e l'API: autenticazione, autorizzazioni, validazione, audit	risponde in millisecondi; non esegue mai operazioni remote in linea
worker Celery	esegue deployment, operazioni di ciclo di vita, discover, riconciliazione	un rilascio dura minuti: bloccare una richiesta HTTP per tutto quel tempo sarebbe fragile
beat	pianifica riconciliazione, controlli di salute e pulizia	separa la pianificazione dall'esecuzione: il worker può essere riavviato senza perdere il calendario
PostgreSQL	stato, piani, passi, audit, configurazioni	unica fonte di verità del control plane
Redis	coda dei job e lock distribuiti	serve un meccanismo veloce e atomico per impedire due operazioni sulla stessa coppia applicazione-host
Volume artefatti	i pacchetti di rilascio caricati	i file non stanno nel database: restano su disco con il loro checksum

4. I livelli del software

La regola è una sola e vale per tutto il codice: **ogni livello parla solo con quello immediatamente sotto**. Una rotta non apre una connessione SSH, un servizio non costruisce un comando `podman`, un adattatore non sa che esiste Flask.



Trasversali a tutti i livelli: sicurezza (cifratura, permessi, validatori, guardia di produzione), audit in sola aggiunta, lock distribuiti, job Celery, repository e modelli.

Figura 3 — La pila dei livelli. Le frecce sono l'unica direzione di dipendenza ammessa.

Il beneficio si vede quando qualcosa va storto. Se un rilascio fallisce su un host, la causa sta in un punto solo della pila e il passo che l'ha incontrata è già registrato con il comando eseguito, l'uscita e il codice di ritorno. Non occorre ricostruire che cosa è stato lanciato: è scritto.

5. Gli oggetti di dominio

Sono definiti in `app/deployment/domain.py` e non dipendono né dal database né dal framework web. È il vocabolario con cui il motore ragiona.

Oggetto	Che cosa rappresenta	Contenuto
<code>DesiredApplicationState</code>	che cosa deve esserci sull'host	applicazione, versione, stato, repliche, immagine, manifest, variabili, porte, volumi, sonda di salute, namespace
<code>ActualApplicationState</code>	che cosa è stato osservato	versione, stato, repliche, salute, dettagli grezzi del runtime
<code>DriftReport</code>	la differenza fra i due	risultato di <code>compute_drift</code> : tipo, dettaglio, presenza
<code>PlanStep</code>	un passo previsto	identificativo, titolo, tipo, parametri, <code>critical</code> , <code>rollback_trigger</code>
<code>DeploymentPlan</code>	il piano completo	riferimento, applicazione, host, runtime, stato desiderato, versione precedente, passi, strategia, rollback automatico
<code>StepExecution</code>	l'esito di un passo	stato, durata, comandi eseguiti, uscita, errore
<code>DeploymentExecution</code>	l'esecuzione completa	la traccia che viene salvata in <code>Deployment</code> e <code>DeploymentStep</code>

La separazione fra **piano** ed **esecuzione** è ciò che rende il sistema ispezionabile. Il piano viene calcolato e salvato come JSON prima di toccare l'host: è possibile leggerlo, confrontarlo con quello di un rilascio precedente e capire che cosa sarebbe successo, anche per un deployment che non è mai partito.

6. Anatomia di un rilascio

Dal wizard alla coda

L'operatore sceglie applicazione, versione e destinazioni. Prima ancora di mettere qualcosa in coda, il servizio verifica i permessi, applica la guardia di produzione, controlla la compatibilità fra manifest e host, cerca conflitti con operazioni in corso ed esegue i controlli preliminari che non richiedono l'host.

Il percorso, dal clic all'host

```
DeploymentService.create()
  risolve applicazione, versione e host; autorizza (prod.* sugli host PROD)
  ProductionGuard.check_operation: frase digitata, motivazione, approvazione
  regole di compatibilit , conflitti, pre-flight statico
  crea DeploymentBatch e Deployment (CREATED > QUEUED oppure PENDING_APPROVAL)
  Celery: run_deployment_batch > deploy_application(id)

DeploymentEngine.execute()
  acquisisce il lock runtime:{host}:{applicazione}
  costruisce DesiredApplicationState da manifest + configurazione + segreti
  DeploymentPlanner.plan() > DeploymentPlan, salvato come JSON
  per ogni passo: transizione di stato, riga DeploymentStep, chiamata all'adattatore
  in caso di errore: stato di fallimento, notifica, rollback automatico se previsto
```

I tredici passi del piano

Il piano   sempre lo stesso nella struttura, e cambia nei dettagli a seconda del runtime, del manifest e del fatto che si tratti di un rilascio o di un ritorno alla versione precedente. I ganci del manifest, quando abilitati, si inseriscono fra i passi previsti.

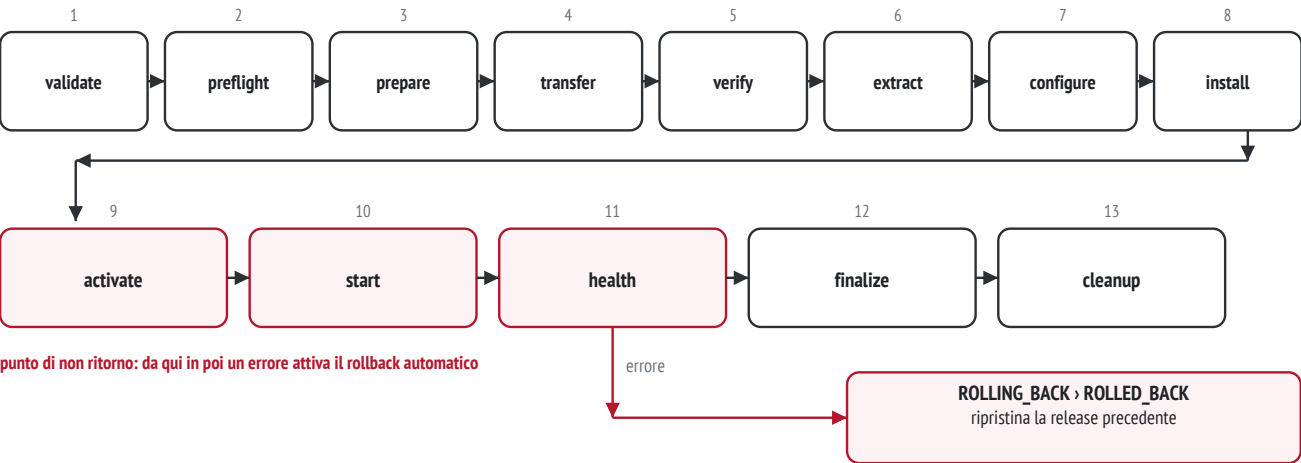


Figura 4 – I passi del piano. Da activate in poi un errore innesca il rollback automatico.

#	Passo	Che cosa fa	Critico
1	validate	verifica il pacchetto e il suo checksum nell'archivio	sì
2	preflight	controlli sull'host: SSH, runtime, disco, memoria, porte, percorso base	sì
3	prepare	crea il layout remoto: releases, shared, staging	sì
4	transfer	invia il pacchetto via SFTP in .part , poi rinomina	sì

#	Passo	Che cosa fa	Critico
5	verify	confronta sha256sum remoto con quello atteso	sì
6	extract	estrae nella cartella della release, senza proprietari né permessi originali	sì
7	configure	scrive shared/config/scarlet.env unendo manifest, configurazione e segreti	sì
–	hook pre_deploy / migrate	script del pacchetto, solo se i ganci sono abilitati per l'applicazione	sì
8	install	carica o scarica l'immagine e crea il container; su Kubernetes applica gli oggetti	sì
9	activate	sposta il symlink current sulla nuova release	sì, innesca il rollback
10	start	avvia l'applicazione con le repliche richieste	sì, innesca il rollback
11	health	esegue la sonda dall'host stesso, con tentativi e intervallo del manifest	sì, innesca il rollback
–	hook post_deploy	script finali del pacchetto	no
12	finalize	aggiorna lo stato effettivo dell'istanza e chiude il deployment	sì
13	cleanup	ripulisce lo staging e le release eccedenti	no

La macchina a stati

Ogni transizione è esplicita e viene scritta prima di essere eseguita. Gli stati di fallimento sono distinti per fase, non generici: dal solo stato finale si capisce dove si è fermato il rilascio.

Fase	Stati di avanzamento	Stato di fallimento
Creazione e approvazione	CREATED, PENDING_APPROVAL, APPROVED, QUEUED	REJECTED, CANCELLED
Validazione	VALIDATING, VALIDATED	VALIDATION_FAILED
Controlli sull'host	PREFLIGHT	PREFLIGHT_FAILED
Trasferimento	TRANSFERRING, TRANSFERRED	TRANSFER_FAILED
Installazione	INSTALLING, INSTALLED	INSTALL_FAILED
Avvio	STARTING, STARTED	START_FAILED
Verifica di salute	HEALTH_CHECKING	HEALTH_CHECK_FAILED
Esito	SUCCESS	ROLLBACK_REQUIRED, ROLLING_BACK, ROLLED_BACK, FAILED

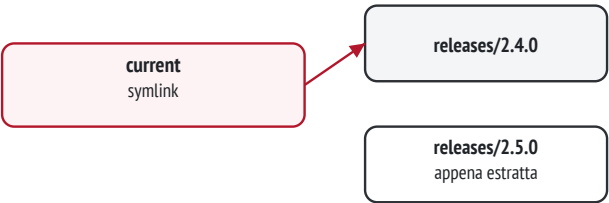
Quando SCARLET riprova

I tentativi automatici esistono, ma solo per gli errori dichiarati ripetibili (connessione SSH caduta, timeout, trasferimento interrotto) e **solo prima che lo stato remoto sia cambiato**. Nessun passo che abbia già modificato l'host viene ripetuto alla cieca.

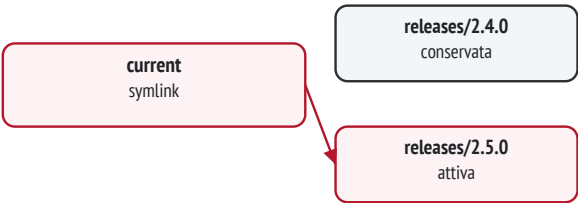
7. Attivazione atomica e rollback

Il momento più delicato di un rilascio è il passaggio dalla vecchia alla nuova versione. SCARLET lo riduce a una singola operazione del filesystem che il kernel garantisce indivisibile.

PRIMA



DOPO



```
ln -sfn ... current.tmp && mv -T ... current.tmp current
```

il rename è atomico: non esiste un istante in cui current non punta a una release valida

il rollback è lo stesso gesto in senso opposto, e la release precedente è già sull'host

Figura 5 – Lo scambio del symlink current: un solo rename, nessuno stato intermedio.

Il vantaggio pratico è che **la versione precedente resta sull'host**, già estratta e pronta. Un rollback non ha bisogno di ritrasferire nulla: rifà gli stessi passi con la release precedente come destinazione, quindi è veloce e non dipende dalla rete né dalla disponibilità del pacchetto. Quante release conservare è un'impostazione di sistema; la corrente e la precedente non vengono mai eliminate.

Situazione	Comportamento
errore prima di activate	il deployment fallisce, l'host resta con la versione precedente attiva e in esecuzione; nessun rollback è necessario
errore in activate , start o health	se il rollback automatico è abilitato e una versione precedente esiste, parte subito: ROLLBACK_REQUIRED › ROLLING_BACK › ROLLED_BACK
rollback richiesto da un operatore	è un deployment a tutti gli effetti, con il proprio riferimento RBK-... , i propri passi e il proprio audit
rollback su PROD	richiede il permesso prod.deployment.rollback e può essere disabilitato globalmente da un'impostazione

8. Adattatori di runtime

Il motore non sa che cosa sia un container. Parla con un'interfaccia di quindici operazioni e riceve l'implementazione giusta da una fabbrica, in base al runtime registrato sull'host.

L'interfaccia `RuntimeAdapter`

```
detect · status · version · inspect · logs · health · install · start
stop · restart · remove · rollback · scale · apply
```

```
RuntimeFactory.get(runtime_type) › DockerRuntimeAdapter | PodmanRuntimeAdapter |
KubernetesRuntimeAdapter
```

	Podman	Docker	Kubernetes
Modalità consigliata	rootless, senza alcun privilegio	demone di sistema, utente nel gruppo docker	API del cluster con kubeconfig (senza SSH), oppure kubectl via SSH
Volumi	montati con <code>:Z</code> per SELinux	montati senza rietichettatura	PersistentVolumeClaim e ConfigMap
Immagini senza rete	<code>podman load -i</code> dall'archivio nel pacchetto	<code>docker load -i</code>	richiede un registry raggiungibile dal cluster
Unità di esecuzione	container con etichette <code>scarlet.*</code>	container con etichette <code>scarlet.*</code>	Deployment, Service, ConfigMap nel namespace
Salute	sonda HTTP o TCP dall'host su <code>127.0.0.1</code>	identica a Podman	stato del Deployment e dei pod
Riavvio dopo il reboot	linger più politica di riavvio, oppure unità systemd	gestito dal demone	gestito dal cluster

Aggiungere un runtime significa scrivere una classe che implementa l'interfaccia e registrarla nella fabbrica. Nessun altro file cambia: né il pianificatore, né la macchina a stati, né le rotte, né l'interfaccia utente.

Il caso particolare: un target senza shell

Un cluster Kubernetes registrato con un kubeconfig non è una macchina in cui SCARLET possa entrare. Non esiste una sessione SSH, non esiste una directory di rilascio, non c'è una chiave host da approvare. È l'unico target che rompe l'assunto implicito di tutto il resto del sistema, e per questo l'accesso passa da un punto solo.

Elemento	Come si comporta su un target cluster
<code>open_target(host)</code>	unico ingresso: restituisce una sessione SSH oppure, per un cluster, un esecutore che rifiuta ogni comando. Un percorso di codice che presupponga ancora una shell fallisce in modo esplicito invece di funzionare a metà
Piano di deployment	senza <code>prepare</code> , <code>transfer</code> , <code>verify</code> , <code>extract</code> , <code>activate</code> , <code>cleanup</code> e senza ganci: resta <code>validate › preflight › configure › install › start › health › finalize</code>
Controlli preliminari	al posto di disco, memoria e porte: API raggiungibile, namespace, permessi della credenziale verificati con una access review, oggetti presenti nel pacchetto
Configurazione	non c'è un file da scrivere su un host: diventa un Secret nel namespace, collegato al container con <code>envFrom</code>
Diagnosi di un errore	l'errore di un rollout porta con sé gli eventi del cluster e lo stato dei pod, che sono ciò che distingue un'immagine non scaricabile da una sonda che non passa

Anche il cluster è dietro una porta, non dietro la libreria ufficiale: l'adattatore parla con un'interfaccia ristretta, implementata sul client reale in esercizio e da un cluster in memoria nei test. È lo stesso schema del livello SSH con il suo client falso, e permette di verificare rilasci, divergenza, rollback e rimozione senza alcun cluster.

9. Il contratto del pacchetto

Fra chi sviluppa le applicazioni e chi le mette in esercizio c'è un contratto scritto e versionato. Un team che lo rispetta produce rilasci che SCARLET sa validare, distribuire, sorvegliare e riportare indietro, senza sapere nulla del funzionamento interno di SCARLET.

Struttura dell'archivio

```
customer-api-2.5.0.scarlet.tar.gz
manifest.yaml          obbligatorio, manifest_version: 1
application/           immagine salvata, binari, file statici
config/               file di ambiente non segreti
scripts/              ganci *.sh richiamati dal manifest
kubernetes/           manifest YAML, per il runtime kubernetes
checksums.sha256       generato dal packager
```

Sezione del manifest	A che cosa serve
application, version, runtime	identità del rilascio; il codice deve coincidere con l'applicazione registrata in SCARLET e la versione è immutabile
image	nome, tag, eventuale archivio dentro il pacchetto e politica di scaricamento (if-not-present, always, never)
ports, volumes	porte pubblicate e volumi persistenti, mappati sull'area shared che sopravvive ai rilasci
environment, environment_files, secrets	variabili non segrete nel pacchetto; i nomi dei segreti sono dichiarati qui e devono essere valorizzati in SCARLET prima del rilascio
healthcheck	tipo (http, https, tcp, command, container_status, kubernetes_status), percorso, porta, tentativi, intervallo
deployment, resources	strategia, timeout, politica di riavvio, utente del container, limiti di CPU e memoria
hooks	script eseguiti sull'host, solo se le operations li hanno abilitati per quell'applicazione
kubernetes, compose	modalità alternative all'immagine singola

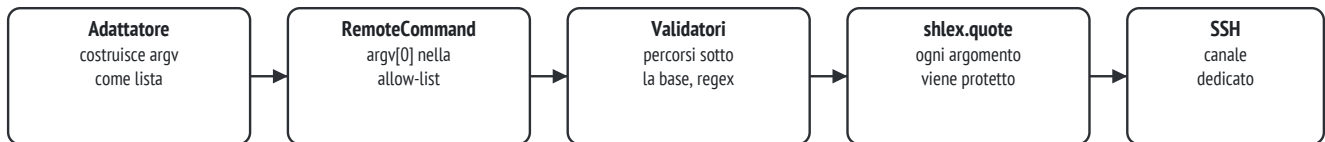
Il pacchetto è dato, non codice

Il validatore ispeziona l'archivio **membro per membro, senza estrarlo**: rifiuta link simbolici e fisici, dispositivi, bit setuid, percorsi assoluti e risalite di directory, e applica limiti al numero di file e alla dimensione decompressa. Un pacchetto malevolo viene respinto prima di toccare il disco, e nessuno script del pacchetto viene mai eseguito sul server SCARLET.

10. Sicurezza per progetto

SCARLET esegue operazioni privilegiate su server di produzione: la postura predefinita è restrittiva e i controlli sono nel codice, non nell'interfaccia.

Come nasce un comando remoto



non esiste una funzione 'esegui questo comando': la shell non viene mai invocata con testo libero

timeout obbligatorio · stdout, stderr, codice di uscita e durata registrati · output ripulito prima del salvataggio

Figura 6 – Il percorso di ogni comando, dalla costruzione all'esecuzione.

Minaccia	Controllo
credenziali SSH sottratte dal database o da un backup	cifratura Fernet con chiave esterna al database; mai esposte da API, interfaccia, log o audit; rotazione per aggiunta
intercettazione sulla connessione SSH	verifica rigida della chiave host; le chiavi sconosciute vanno approvate da un amministratore; una difformità blocca l'host e genera un evento CRITICAL
iniezione di comandi attraverso nomi, versioni, manifest, filtri	comandi come lista di argomenti, <code>shlex.quote</code> su ognuno, allow-list degli eseguibili, espressioni regolari su ogni valore che finisce in un comando
pacchetto di rilascio malevolo	ispezione membro per membro senza estrazione, rifiuto di link e file speciali, limiti di numero e dimensione, schema del manifest rigido
modifica non autorizzata in produzione	permessi <code>prod.*</code> , motivazione obbligatoria, frase da digitare, approvazione di una seconda persona, lock per applicazione e host
scalata di privilegi dall'interfaccia	RBAC applicato in ogni rotta e in ogni servizio; nascondere un pulsante è solo cosmesi
attacchi web (XSS, CSRF, clickjacking)	autoescape, CSP senza script in linea, token CSRF, <code>frame-ancestors 'none'</code> , cookie Secure, HttpOnly e SameSite
forza bruta sulle credenziali	limite di frequenza sul login, blocco dell'account, Argon2id, messaggi di errore identici
esecuzione arbitraria da parte degli operatori	non esiste una funzione per eseguire comandi liberi; la shell diagnostica è disabilitata e rifiutata in configurazione di produzione

Controlli sulla produzione

Un host marcato come produzione cambia le regole per ogni operazione che lo riguarda. Il permesso di base non basta: serve anche la sua variante `prod.*`. A questo si aggiungono i controlli configurabili a livello di sistema, e vale sempre il più restrittivo fra impostazione globale e regola dell'ambiente.

Controllo	Effetto
Permesso <code>prod.<permesso></code>	OPERATOR può agire solo su DEV; PROD_OPERATOR e ADMIN anche su PROD
Frase digitata	l'operatore deve scrivere una frase esatta, configurabile, per confermare l'operazione
Motivazione obbligatoria	il testo finisce nell'audit e nella notifica, accanto al nome di chi ha agito
Approvazione di una seconda persona	il deployment resta in <code>PENDING_APPROVAL</code> ; chi lo ha richiesto non può approvarlo
Lock per applicazione e host	impedisce due operazioni contemporanee sulla stessa coppia, anche da sessioni diverse

11. Riconciliazione e divergenza

Il riconciliatore è il motivo per cui SCARLET sa dirvi che cosa sta girando davvero sui server, e non soltanto che cosa è stato distribuito l'ultima volta.



Figura 7 – Il ciclo di riconciliazione: osserva, confronta, segnala.

Aspetto	Comportamento predefinito
Frequenza	ogni 5 minuti, modificabile da Sistema > Impostazioni
Ambito	tutti gli host abilitati; gli host disabilitati sono esclusi
Effetto normale	aggiorna lo stato effettivo e la divergenza, senza toccare l'host
Rimedio automatico	disattivato; abilitabile e comunque limitato agli ambienti non di produzione
Visibilità	riquadro delle divergenze sul cruscotto, scheda dell'applicazione, notifiche

Accanto alla riconciliazione lavora il controllo di salute periodico, che interroga la sonda dichiarata nel manifest e conserva lo storico degli esiti. Le due informazioni sono distinte: un'applicazione può essere in esecuzione nella versione giusta e comunque rispondere male alla sonda.

12. Job asincroni, code e lock

Tutto ciò che tocca un host remoto passa da un job. La richiesta HTTP si limita a validare, autorizzare, creare le righe e mettere in coda: restituisce un identificativo e termina.

Coda	Che cosa contiene
<code>scarlet</code>	operazioni di ciclo di vita e operazioni sugli host: avvio, arresto, riavvio, stato, log, test di connessione, discover
<code>scarlet-deploy</code>	deployment e rollback: sono i job lunghi, tenuti separati per non fare da tappo alle operazioni brevi
<code>scarlet-maintenance</code>	riconciliazione, controlli di salute, pulizia dei dati e delle release remote

Meccanismo	Come funziona
Lock distribuito	chiave <code>runtime:{host}:{applicazione}</code> in Redis con <code>SET NX PX</code> e rilascio via script Lua; ripiego sul database se Redis non è disponibile
Tentativi	attesa esponenziale, solo per errori dichiarati ripetibili e solo prima che lo stato remoto sia cambiato
Timeout	ogni comando remoto ne ha uno; il trasferimento dei file ne ha uno proprio, più generoso
Idempotenza	i passi che hanno già prodotto il risultato atteso vengono saltati, ad esempio la release già presente durante un rollback

13. Modello dei dati

Il database è la sola fonte di verità del control plane. Le tabelle sono raggruppate per responsabilità; le migrazioni sono versionate e applicate all'avvio.

Gruppo	Tabelle	Note
Utenti e permessi	users, roles, permissions, role_permissions, user_roles, api_tokens	password con Argon2id, token con impronta SHA-256
Infrastruttura	environments, target_hosts, target_credentials, ssh_keys, host_groups, host_group_members, runtime_capabilities	le credenziali sono cifrate a riposo
Applicazioni	applications, application_versions, packages, application_instances	le versioni sono immutabili; le istanze tengono desiderato ed effettivo
Operazioni	deployment_batches, deployments, deployment_steps, deployment_approvals, lifecycle_operations, operation_logs, health_checks, distributed_locks	il piano è salvato come JSON sul deployment
Governo	audit_logs, security_events, configurations, configuration_entries, configuration_versions, notifications, system_settings	l'audit è in sola aggiunta

Gli indici seguono le interrogazioni reali: deployment per host, per applicazione, per data e per stato; audit per data, utente e azione; operazioni per stato e data; versioni per applicazione; istanze per host e applicazione.

14. Osservabilità e tracciabilità

Strumento	A che cosa serve
Identificativo di richiesta (ULID)	generato a ogni richiesta, restituito nell'intestazione X-Request-ID , riportato nei log JSON, nelle righe di audit e nel contesto del worker: lega il clic dell'operatore al comando eseguito sull'host
Riferimenti leggibili	DEP-AAAAMMGG-nnnnnn per i deployment, RBK-... per i rollback, OP-... per le operazioni, BATCH-... per i lotti
Audit in sola aggiunta	chi, che cosa, su quale oggetto, con quale esito e con quale motivazione; non modificabile dall'applicazione
Eventi di sicurezza	login falliti, blocchi, difformità di chiave host, operazioni negate, con livello di gravità
Log dei passi	per ogni passo: comando, uscita standard, errori, codice di uscita e durata, ripuliti dai segreti prima del salvataggio
Endpoint di servizio	/api/health (vitalità), /api/ready (database, Redis, worker), /api/metrics (Prometheus)

La conseguenza pratica: davanti a un incidente si parte dal riferimento del deployment e si arriva, senza salti, al comando esatto eseguito sull'host, all'ora precisa, per conto di quale utente e con quale motivazione dichiarata.

15. Punti di estensione

L'architettura è stata pensata per essere allargata nei punti in cui è prevedibile che serva, senza rimettere mano al motore.

Estensione	Come si realizza
Nuovo runtime	implementare <code>RuntimeAdapter</code> e registrarlo in <code>RuntimeFactory</code>
Archivio artefatti diverso	implementare <code>ArtifactStorage</code> (ad esempio S3 o MinIO)
Segreti da un vault esterno	implementare <code>SecretProvider</code> (Vault, AWS Secrets Manager, Secret di Kubernetes)
Nuovo ambiente oltre DEV e PROD	inserire una riga in <code>environments</code> : il modello non è limitato a due
Approvazioni a più firme	le righe <code>DeploymentApproval</code> e lo stato <code>PENDING_APPROVAL</code> esistono già; si estende <code>ProductionGuard.check_approval</code>
Strategie di rilascio	<code>DeploymentStrategy.CANARY</code> è riservata; il lotto gestisce già sequenziale e parallelo
Aggiornamenti in tempo reale	oggi l'interfaccia interroga periodicamente gli endpoint dei passi; i passi sono salvati a ogni cambiamento, quindi un canale WebSocket o SSE si aggiunge senza toccare il motore

Questo documento è generato dal repository con `python scripts/build_pdf_docs.py` e segue `docs/architecture.md`, `docs/security.md` e il codice del motore di deployment.